Practical Two-Factor Authentication (2FA) Implementation using TOTP

2FA Practical Exercise: Implement Time-based One-Time Password (TOTP) two-factor authentication on a web server or user portal.

Introduction

Two-Factor Authentication (2FA) is a security method that requires users to verify their identity using two authentication factors. The first factor is something the user knows, such as a password, and the second factor is something the user has, such as a Time-based One-Time Password (TOTP) generated by an authenticator app. TOTP codes change every 30 seconds, providing an additional layer of security. Implementing TOTP-based 2FA helps protect user accounts from unauthorized access even if the password is compromised.

Objectives

Aim: To integrate Time-based One-Time Password (TOTP) based Two-Factor Authentication (2FA) into a user authentication system. Generate and register a secure TOTP secret using an authenticator application such as Google Authenticator. Verify the 6-digit authentication code during login to enhance account security and prevent unauthorized access. Generate cryptographically random TOTP secret keys and QR codes.

- Bind dynamic authenticator apps (Google/Microsoft Authenticator) to user accounts.
- Validate 6-digit TOTP verification codes during authentication.

Requirements

- **Python/Node.js environment or Linux system:** A Linux-based operating system (such as Kali Linux or Ubuntu) with Python or Node.js installed to develop and execute the TOTP authentication application.
- **TOTP library (pyotp / speakeasy) or Keycloak MFA engine:** A software library or authentication platform used to generate, manage, and validate Time-based One-Time Passwords for two-factor authentication.
- **Mobile Authenticator app:** An authenticator application such as Google Authenticator or Microsoft Authenticator installed on a smartphone to scan the QR code and generate 6-digit TOTP verification codes.

Expected Output

- Generated secret seed and rendered QR code.
- Successful authentication log verifying 2FA token match within 30-second time window.

Step by Step Procedure(s)

Step 1: Install Required Python Packages

Update the package repository to ensure the latest software versions are available on the system. Install the Python package manager (pip) if it is not already present. Using pip, install the required libraries pyotp and qrcode, which are used for generating TOTP secrets and QR codes. These packages provide the functionality required for implementing Two-Factor Authentication (2FA). Verify that the installation completes successfully before proceeding.

Fig 4.1: update system, install python and qr generator

Step 2: Create the TOTP Authentication Script

Create a new Python file named totp.py using a text editor such as Nano or VS Code. Copy and paste the provided Python code into the file without making any modifications. The script generates a unique Base32 secret key, creates a QR code containing the TOTP configuration, and saves it as totp_qr.png. It also prompts the user to enter the OTP generated by the authenticator application and verifies the entered code. Save the file after adding the code.

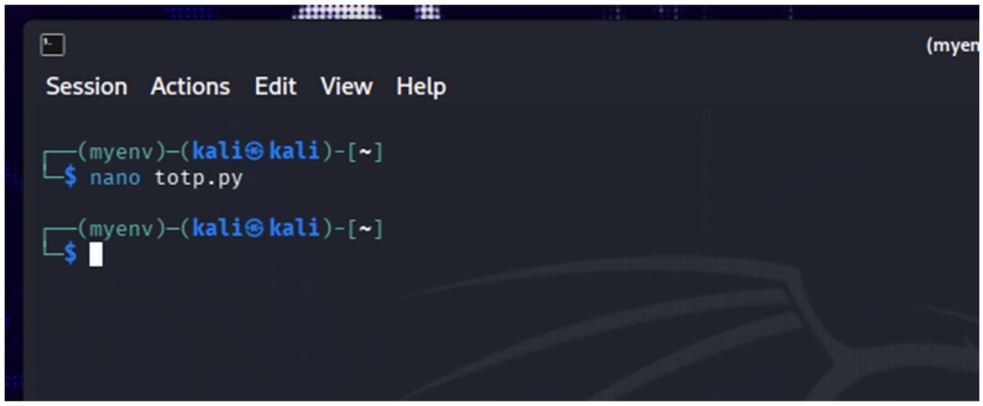

Fig 4.2: edit totp file

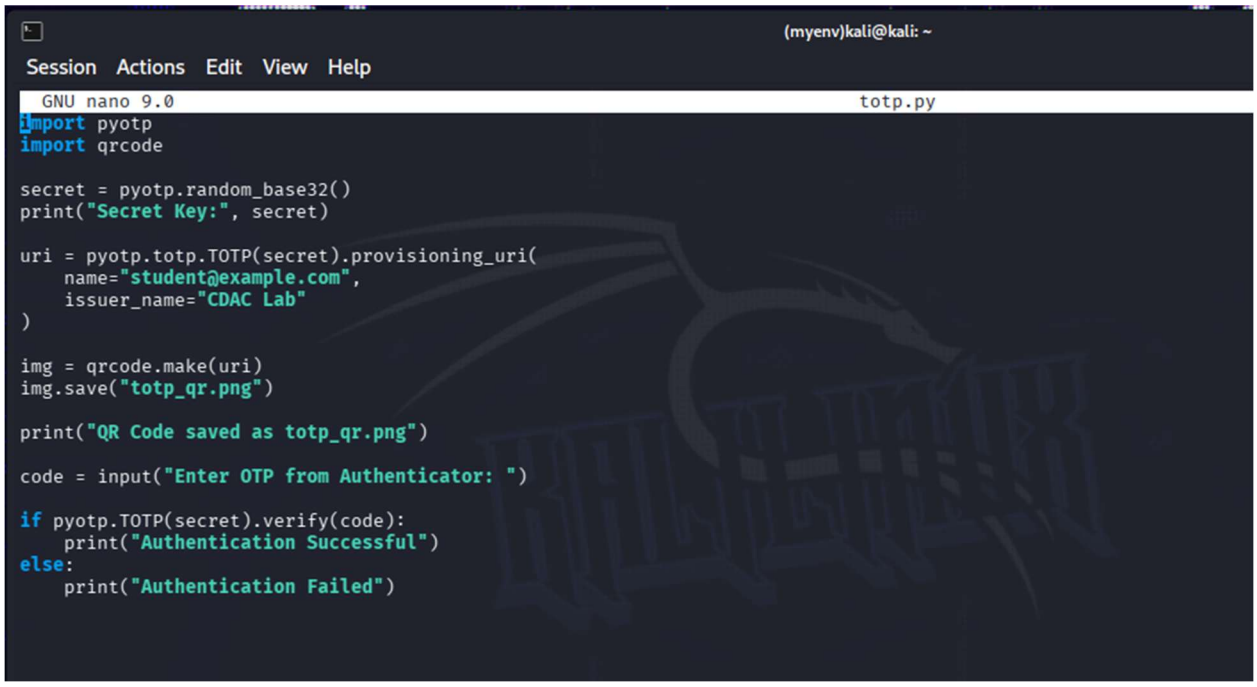

Fig 4.3: paste the python code

Step 3: Execute the Python Program

Run the Python script using the Python interpreter from the terminal. After successful execution, the program displays the generated secret key and automatically creates a QR code image named totp_qr.png in the current working directory. This QR code contains the TOTP information required by the authenticator application. Ensure that the QR code file has been created successfully before continuing to the next step. Keep the terminal window open for OTP verification.

Fig 4.4: a qr file will be saved in program path

Step 4: Register the Authenticator and Verify the OTP

Install the Google Authenticator application on your mobile device from the Play Store or App Store. Sign in with your Google account if required and choose the option to add a new account by scanning a QR code. Scan the generated totp_qr.png image displayed on your computer. The application will immediately begin generating a new 6-digit OTP that refreshes every 30 seconds. Enter the displayed OTP into the terminal when prompted, and the program will validate it and display "Authentication Successful" if the code is correct.

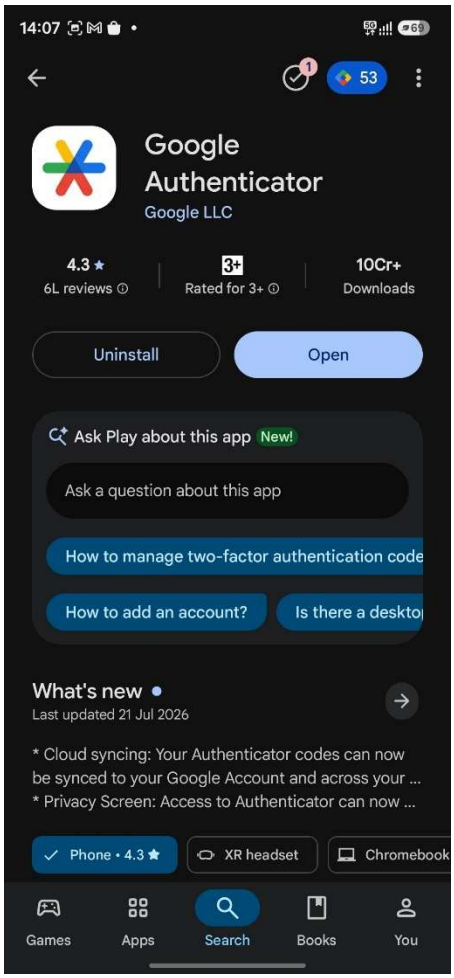

Fig 4.5: Install Google Authenticator

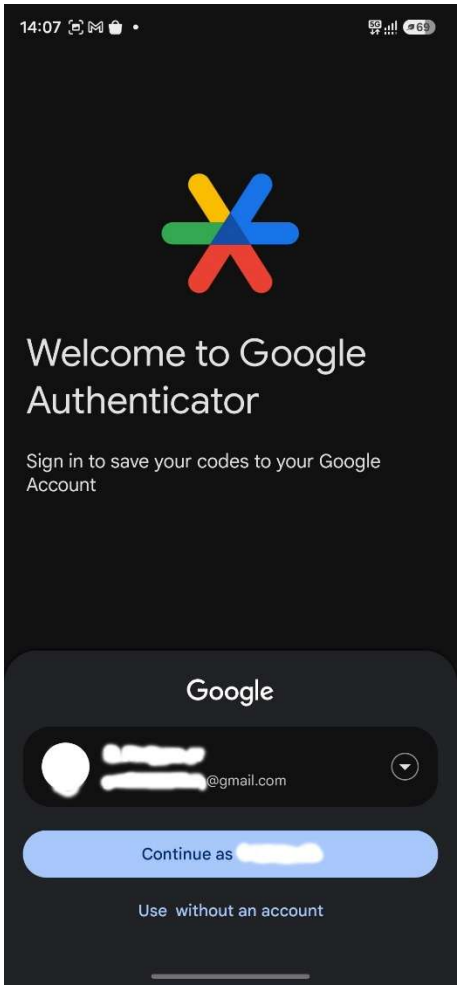

Fig 4.6: Sign in with Gmail

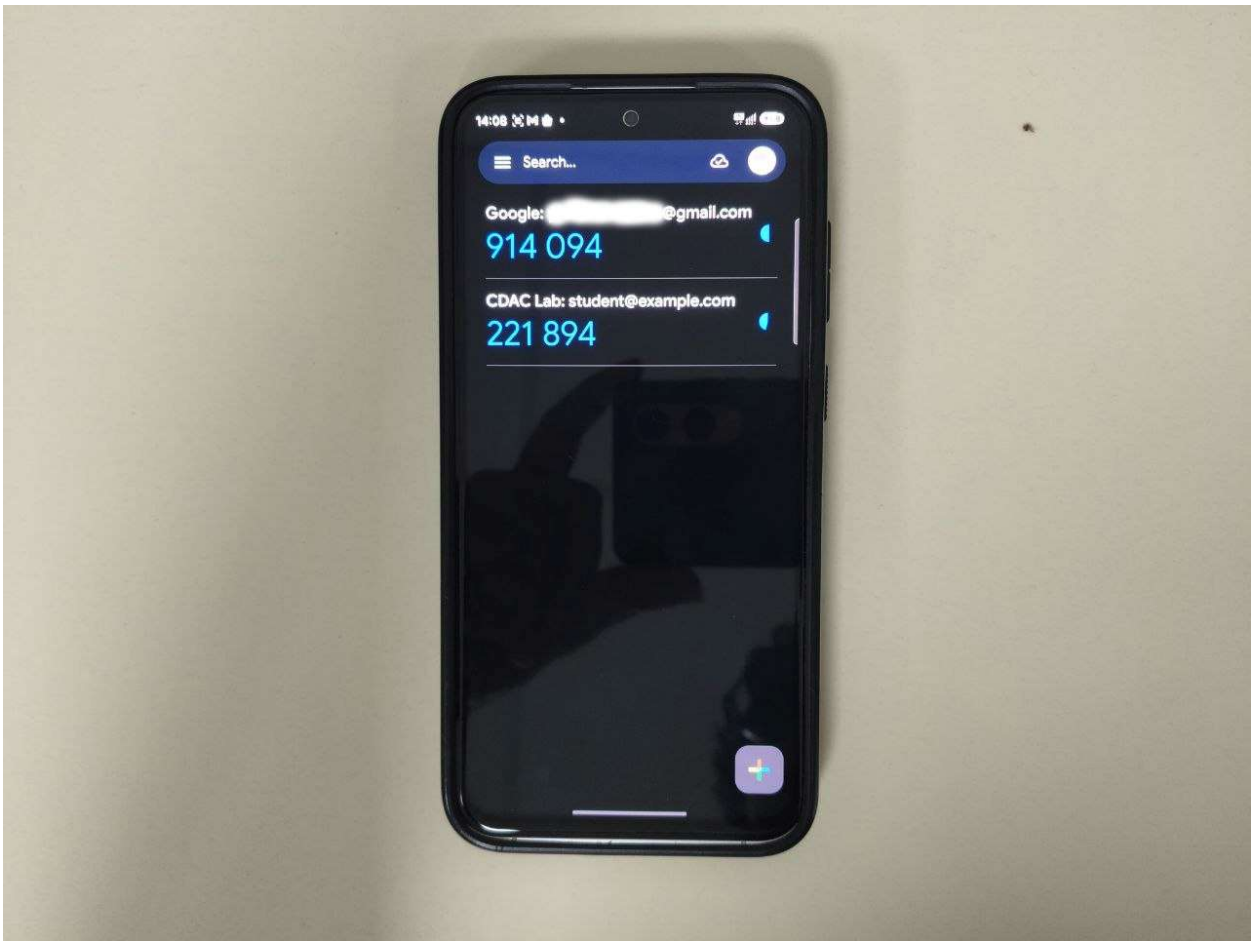

Fig 4.7: OTP Will be generated in Google Authenticator after Scanning QR Code

Fig 4.8: enter OTP and the Authentication will be successful

Learning Outcomes

- Successfully implemented Time-based One-Time Password (TOTP) based Two-Factor Authentication (2FA) using Python and the pyotp library.
- Learned how to generate secure Base32 secret keys and create QR codes for registering users with an authenticator application.
- Understood the process of configuring Google Authenticator to generate dynamic 6-digit verification codes.
- Performed practical server-side validation of TOTP codes within the standard 30-second authentication window.
- Gained hands-on experience with integrating an additional authentication factor into a user login process.
- Understood the role of cryptographic secret keys and time synchronization in secure OTP generation and validation.
- Observed how TOTP-based authentication strengthens account security by reducing the risk of password compromise and replay attacks.
- Developed practical knowledge of implementing industry-standard multi-factor authentication techniques for securing web applications and user accounts.

Notes/Observations

- A unique Base32 secret key and QR code were successfully generated using the pyotp library.
- The QR code was scanned using Google Authenticator to register the user account.
- The authenticator app generated a new 6-digit OTP every 30 seconds.
- Authentication succeeded only when the correct OTP was entered within the valid time window.
- Invalid or expired OTPs were rejected, ensuring secure user verification.
- Time synchronization between the server and the authenticator app is essential for accurate OTP validation.
- The practical demonstrated that TOTP-based 2FA significantly enhances login security by adding an extra authentication factor.